

Máster Ingeniería de Telecomunicación

Sistemas Ciberfísicos y Seguridad Hardware Memoria del proyecto

Autores:

Manuel Fernández Giráldez

Daniel González Domínguez

Nuria Sorrentino Rebull

2º Máster en Ingeniería de Telecomunicación
Universidad de Sevilla



Sistemas Ciberfísicos y Seguridad Hardware

Memoria del proyecto

Autores:

Manuel Fernández Giráldez

Nuria Sorrentino Rebull

Daniel González Dominguez

2º Máster en Ingeniería de Telecomunicación

Universidad de Sevilla

Resumen

Este proyecto implementa un sistema IoT sobre la placa STM32L475 (B-L475E-IOT01A) usando FreeRTOS, conectividad WiFi y mensajería MQTT. El sistema está diseñado para funcionar como una red de nodos lógicos: en la versión actual se trabajan dos roles principales (Nodo ACC para acelerómetro y Nodo ENV para temperatura/humedad), pero la arquitectura está planteada para ser escalable a N nodos, manteniendo el mismo firmware y diferenciando comportamiento por identificador de nodo y por topics MQTT parametrizados.

La configuración del sistema se realiza de dos formas complementarias:

- Configuración en compilación mediante `NODE_ID`, que define el rol del nodo (ACC/ENV) y el sufijo de sus topics (`bridge/.../<id>`).
- Configuración en ejecución mediante comandos por UART o MQTT (topics `bridge/cmd/<id>`), permitiendo cambiar parámetros sin reflashear: credenciales WiFi (`WIFI <ssid> <pass>`), ajuste de reloj (RTC ...), forzar capturas (READ) y control de modos (`CONT ON/OFF`).

La arquitectura se divide en tareas concurrentes: una tarea de WiFi (conexión y reintentos), una tarea de MQTT (sesión, suscripción a `bridge/cmd/<id>` y publicación desde una cola), una tarea de UART (parsing de comandos), y una tarea de aplicación específica por rol (ACC o ENV). Los datos se publican en topics dedicados (`bridge/accel/<id>`, `bridge/env/<id>`) y, opcionalmente, los logs se pueden publicar en `bridge/log/<id>`, manteniendo la depuración tanto local (UART/SWV) como remota (MQTT).

A nivel de coordinación entre nodos, el Nodo ENV puede emitir comandos hacia el Nodo ACC publicando en `bridge/cmd/1` para activar/desactivar el modo continuo del acelerómetro cuando se detecta una condición de alerta, demostrando un esquema de control distribuido basado en MQTT que puede extenderse de forma natural a más nodos simplemente asignando nuevos IDs y reutilizando la misma estructura de topics y comandos.

Resumen	5
Índice	7
1 Descripción del proyecto	9
1.1 <i>Requisitos Obligatorios</i>	9
1.1.1 Arquitectura del Sistema	9
1.1.2 Reloj en Tiempo Real (RTC)	9
1.1.3 Puertos de Comunicación	9
1.1.4 Monitorización de Sensores	10
1.1.5 Conectividad	10
1.1.6 Procesamiento y Visualización	11
1.2 <i>Requisitos Opcionales Implementados</i>	11
1.2.1 Configuración Dinámica del RTC	11
1.2.2 Configuración Dinámica de WiFi	11
1.2.3 Sistema Unificado de Comandos (UART + MQTT)	11
1.2.4 Sistema de Logging Dual	12
1.3 <i>Justificación de Decisiones de Diseño</i>	12
2 Diseño del sistema	13
2.1 <i>Diseño de tareas</i>	13
2.1.1 Asignación de Prioridades	13
2.1.2 Dimensionamiento de Stacks	13
2.2 <i>Diseño de Comunicaciones</i>	14
2.2.1 Comunicación Interna (Local al Nodo)	14
2.2.2 Optimización de Memoria en Cola MQTT	14
2.2.3 Comunicación Externa (Red)	15
2.2.4 Formato de mensajes	15
2.3 <i>Topics MQTT del sistema</i>	17
3 Implementación	18
3.1 <i>Gestión del WiFi</i>	18
3.1.1 Tarea de gestión WiFi (WifiTask)	18
3.2 <i>MQTT</i>	19
3.3 <i>Tarea de gestion de MQTT (MQTT_TaskFun)</i>	19
3.3.1 Fase 1: Espera de Conectividad WiFi	19
3.3.2 Fase 2: Conexión TCP al Broker	20
3.3.3 Fase 3: Handshake MQTT y Suscripción	20
3.3.4 Fase 4: Bucle de Publicación y KeepAlive	20
3.3.5 Fase 5: Limpieza y Reinicio del Ciclo	21
3.4 <i>Configuración por UART y MQTT. Parseo de comandos</i>	22
3.4.1 Tarea de configuración por UART (UartCfgTask_Func)	22
3.4.2 Recepción remota por MQTT y unificación del parseo	23
3.4.3 Distribución y ejecución en tareas de aplicación	23
3.5 <i>NODO 1: Acelerómetro y FIFO</i>	24
3.5.1 Tareas y funciones	24
3.5.2 Funcionamiento	26
3.6 <i>NODO 2: Temperatura y humedad</i>	27

3.6.1	Tareas y funciones	28
3.7	<i>Logs por UART y MQTT</i>	29
3.7.1	Salida local: UART/SWV y control de concurrencia	29
3.7.2	Uso de Mutex	29
3.7.3	Publicación remota: logs en MQTT sin realimentación	30
4	Pruebas y validación	30
4.1	<i>Metodología de Desarrollo Incremental</i>	30
4.1.1	Fase 1: Infraestructura de Comunicaciones (WiFi + MQTT)	30
4.1.2	Fase 2: Integración de Sensores (Desarrollo Paralelo)	30
4.2	<i>Debugging de Hard Fault por Stack Overflow</i>	31
4.2.1	Detección e Identificación	31
4.2.2	Solución Implementada: Refactorización con Punteros	32
5	Interfaz de usuario	34
6	Conclusiones y trabajos futuros	35
6.1	<i>Conclusiones</i>	35
6.2	<i>Trabajos futuros</i>	35

Índice de tablas

Tabla 1: tareas	13
Tabla 2: campos del mensaje del acelerómetro	16
Tabla 3: campos de mensaje del nodo de medición ambiental	16

Índice de figuras

Figura 1: Diagrama de flujo de MQTT_Task	21
Figura 2. Diagrama de flujo UartCfgTask_Fun	23
Figura 3. Diagrama de flujo envRead_Task_Func.	28
Figura 4. JSON generado.	29
Figura 5: algunos logs de la ejecución del nodo Acelerómetro	33
Figura 6. Interfaz gráfica.	34

1 DESCRIPCIÓN DEL PROYECTO

El desarrollo del sistema de monitorización estructural se ha realizado siguiendo estrictamente las especificaciones definidas en la memoria del proyecto, diferenciando entre requisitos obligatorios (implementación mínima) y requisitos opcionales (ampliaciones funcionales). Esta sección documenta el conjunto completo de requisitos contemplados y su grado de cumplimiento en la implementación final.

1.1 Requisitos Obligatorios

Los requisitos obligatorios definen las funcionalidades mínimas que el sistema debe proporcionar para cumplir con los objetivos del proyecto. La implementación desarrollada satisface la totalidad de estos requisitos, tal como se detalla a continuación.

1.1.1 Arquitectura del Sistema

El sistema se compone de dos nodos distribuidos con funciones especializadas: el nodo tipo 1 se dedica a la monitorización de aceleraciones estructurales, mientras que el nodo tipo 2 captura condiciones ambientales (temperatura y humedad relativa). Esta distribución de responsabilidades permite especializar cada nodo en su sensor específico, optimizando el uso de recursos y facilitando el escalado futuro del sistema a más nodos sin modificaciones arquitectónicas.

Ambos nodos ejecutan FreeRTOS en modo preemptive, garantizando que las tareas de mayor prioridad puedan interrumpir a las de menor prioridad en caso necesario. La arquitectura multitarea implementada consta de cinco tareas especializadas (WifiTask, MQTT_Task, Accel_Task o task_envRead según tipo de nodo, y UartCfgTask), coordinadas mediante dos colas de mensajes (qMqttTx para publicación de datos y qCmdRx para recepción de comandos). El uso de colas permite el desacoplamiento temporal entre productores y consumidores de información, evitando bloqueos y garantizando que cada tarea opere a su frecuencia natural.

El sistema hace uso extensivo de notificaciones de FreeRTOS (thread flags) para sincronizar eventos asíncronos con las tareas de aplicación. Se han implementado cuatro tipos de notificaciones: NOTE_BUTTON_IRQ (pulsación del botón de usuario), NOTE_RTC_WAKEUP (alarma periódica del RTC), NOTE_ACCEL_FIFO (FIFO watermark del acelerómetro), y NOTE_UART_LINE (línea completa recibida por UART). Este mecanismo permite que las interrupciones hardware despierten tareas bloqueadas con latencia mínima y sin consumo de CPU durante los periodos de inactividad.

1.1.2 Reloj en Tiempo Real (RTC)

El RTC se ha configurado con alarma periódica programada para dispararse cada 30 minutos, cumpliendo con el requisito de monitorización periódica automática. La alarma genera una interrupción que, mediante el callback HAL_RTC_AlarmAEventCallback(), despierta la tarea de sensor correspondiente (compilación condicional según NODE_ID). Esta estrategia permite que el sistema entre en modos de bajo consumo entre capturas, despertándose únicamente cuando es necesario realizar una medición.

El RTC también proporciona los timestamps asociados a cada muestra adquirida, garantizando la trazabilidad temporal de las mediciones para su posterior análisis espectral y correlación entre nodos.

1.1.3 Puertos de Comunicación

Se han implementado dos puertos UART diferenciados funcionalmente:

1. Puerto de depuración: Proporciona trazas de ejecución en tiempo real mediante la función dbg_printf(), que imprime mensajes tanto por UART1 como por SWV (Serial Wire Viewer). Este puerto permite verificar el funcionamiento del sistema durante el desarrollo y debugging sin necesidad de herramientas externas.

2. Puerto de configuración: Gestionado por la tarea `UartCfgTask`, permite enviar comandos de control al sistema (configuración de WiFi, ajuste de RTC, disparo manual de lecturas, etc.). Los comandos se procesan mediante el módulo `cmd_parser.c`, que valida la sintaxis y genera estructuras de comando que son distribuidas a las tareas correspondientes.

1.1.4 Monitorización de Sensores

1.1.4.1 Acelerómetro (Nodo 1)

- El sensor LSM6DSL se ha configurado cumpliendo estrictamente las especificaciones:
- Frecuencia de muestreo (ODR): 52 Hz
- Rango de escala completa: $\pm 2g$
- Eje monitorizado: Z (vertical, perpendicular al plano del puente)
- Resolución de salida: mili g (mg)

Se utiliza la interrupción FIFO watermark en lugar de la simple "Data Ready", mejorando significativamente la eficiencia del sistema. El FIFO hardware del LSM6DSL puede almacenar hasta 512 muestras; configurando un watermark de 64 muestras, la interrupción solo se dispara cada 1.23 segundos (64/52Hz), reduciendo drásticamente la carga de procesamiento de interrupciones en comparación con una IRQ por muestra.

El sistema implementa dos modos de operación diferenciados:

- Modo normal: Captura 64 muestras por evento de disparo (botón, RTC, comando). Optimizado para verificaciones periódicas con bajo consumo de ancho de banda de red.
- Modo continuo: Captura 1024 muestras y las transmite en ráfaga. Diseñado para análisis modal detallado cuando se detectan condiciones de alarma.

Cada conjunto de muestras se acompaña de su timestamp de inicio, permitiendo reconstruir la base de tiempo exacta en el receptor conociendo la frecuencia de muestreo.

1.1.4.2 Sensor Ambiental (Nodo 2)

El sensor HTS221 captura temperatura y humedad relativa con las siguientes características:

- Temperatura: Transmitida en grados Celsius con 1 decimal (formato: `valor×10` para evitar punto flotante en JSON)
- Humedad: Transmitida en porcentaje sin decimales
- Período de muestreo: 30 minutos (alarma RTC)
- Disparos adicionales: Botón de usuario (permite forzar lecturas on-demand)

1.1.5 Conectividad

La comunicación inalámbrica se implementa mediante WiFi 802.11 b/g/n usando el módulo ISM43362 integrado en la placa. La tarea `WifiTask` gestiona la conexión al punto de acceso, con capacidad de reconexión automática ante caídas del enlace.

El protocolo MQTT v3.1.1 actúa como capa de aplicación, conectando con brokers públicos (`test.mosquitto.org`, `broker.emqx.io`) en el puerto estándar 1883 (sin TLS en esta versión). El sistema se suscribe al topic de control `bridge/cmd/<NODE_ID>` para recibir comandos remotos, mientras publica datos de sensores en topics jerárquicos diferenciados:

- `bridge/accel/1`: Datos del acelerómetro (nodo 1)
- `bridge/env/2`: Datos ambientales (nodo 2)
- `bridge/log/1`, `bridge/log/2`: Logs del sistema

Esta segregación de topics facilita la escalabilidad y permite que múltiples instancias de la aplicación de supervisión se suscriban selectivamente a los datos de interés sin interferencias.

1.1.6 Procesamiento y Visualización

El procesamiento de señales se realiza en una aplicación Python ejecutada en PC, que implementa:

1. Cálculo de FFT (Fast Fourier Transform): Analiza el espectro frecuencial de las aceleraciones para detectar las frecuencias naturales de vibración de la estructura. Se utiliza NumPy (`np.fft.rfft()`) para obtener una FFT real optimizada, eliminando previamente la componente DC (media) de la señal.
2. Detección de picos espectrales: Identifica automáticamente las frecuencias dominantes en el espectro mediante `np.argmax()`, anotándolas gráficamente si superan un umbral de magnitud configurable.
3. Interfaz gráfica de usuario (GUI): Desarrollada con Tkinter y Matplotlib, presenta 6 gráficas en tiempo real organizadas en una matriz 2×3:
 - Fila superior: Señales en dominio temporal (Temperatura, Humedad, Aceleración Z)
 - Fila inferior: Espectros de frecuencia de la aceleración
4. Sistema de alarma automática: Monitoriza continuamente la temperatura recibida del nodo ambiental. Si supera un umbral configurable, publica automáticamente el comando CONT ON en los topics `bridge/cmd/1` y `bridge/cmd/2`, activando el modo de monitorización continua en ambos nodos. Cuando la temperatura normaliza, envía CONT OFF para retornar al modo normal.

1.2 Requisitos Opcionales Implementados

Además de la funcionalidad mínima obligatoria, se han incorporado varias ampliaciones opcionales que mejoran significativamente la usabilidad, flexibilidad y capacidad de diagnóstico del sistema.

1.2.1 Configuración Dinámica del RTC

Se ha implementado la capacidad de ajustar la fecha y hora del RTC mediante comandos tanto por UART como por MQTT, utilizando el formato:

```
RTC <año> <mes> <día> <hora> <minuto> <segundo>
```

Esta funcionalidad elimina la necesidad de recompilar el firmware para sincronizar los relojes de los nodos con el tiempo real. El comando se procesa mediante la función `ApplyRTC()`, que configura el hardware RTC del STM32 y confirma la operación mediante un log en el topic `bridge/log/<NODE_ID>`.

La posibilidad de sincronizar los RTC de forma remota vía MQTT es especialmente valiosa para nodos desplegados en ubicaciones de difícil acceso, ya que permite corregir derivas temporales sin intervención física.

1.2.2 Configuración Dinámica de WiFi

Las credenciales de la red WiFi (SSID y contraseña) pueden modificarse en tiempo de ejecución mediante el comando:

```
WIFI <ssid> <password>
```

Al recibir este comando (por UART o MQTT), el sistema almacena las nuevas credenciales en las variables globales `g_wifi_ssid` y `g_wifi_pass`, y fuerza una reconexión con los nuevos parámetros. Esto permite reubicar los nodos entre diferentes redes sin necesidad de reprogramar el firmware, facilitando despliegues en ubicaciones con infraestructuras WiFi ya existentes.

1.2.3 Sistema Unificado de Comandos (UART + MQTT)

Una de las implementaciones opcionales más destacables es la transparencia de canal de comandos. El sistema acepta los mismos comandos textuales tanto por consola UART local como por mensajes MQTT remotos. El flujo de procesamiento es idéntico en ambos casos:

1. El comando llega como cadena de texto (desde `HAL_UART_RxCpltCallback()` o desde el callback `MQTT_OnMqttControlMessage()`).
2. Se parsea mediante `Cmd_ParseLine()`, que valida la sintaxis y genera una estructura `SystemCommandMsg_t`.
3. Se encola en `qCmdRx` con la fuente identificada (`CMD_SRC_UART` o `CMD_SRC_MQTT`).
4. La función `DispatchCommand()` despierta la tarea de aplicación correspondiente según el tipo de nodo.

Esta arquitectura unificada simplifica el mantenimiento del código (un único parser, una única cola) y proporciona redundancia: si el canal MQTT falla, el sistema sigue siendo configurable por UART local.

1.2.4 Sistema de Logging Dual

Se ha desarrollado un sistema de logging con dos niveles de publicación:

- `dbg_printf()`: Logs técnicos de bajo nivel (direcciones MAC, IPs asignadas, estados internos de conexión). Se imprimen únicamente por UART/SWV, evitando saturar el broker MQTT con información de debugging que solo es relevante durante el desarrollo.
- `log_printf()`: Eventos de aplicación de alto nivel (inicialización de sensores, comandos ejecutados, errores de configuración). Se imprimen tanto por UART como por MQTT (topic `bridge/log/<NODE_ID>`), permitiendo supervisión remota del estado de los nodos sin acceso físico.

Ambas funciones utilizan mutex de FreeRTOS para garantizar exclusión mutua en el acceso al puerto UART, evitando entrelazado de caracteres cuando múltiples tareas intentan imprimir simultáneamente. Adicionalmente, `log_printf()` implementa un filtro antifeedback que detecta mensajes que contienen "bridge/log/" en su payload, evitando bucles infinitos (log de un mensaje MQTT que genera otro log que se publica y vuelve a generar un log, etc.).

1.3 Justificación de Decisiones de Diseño

Uso de FIFO watermark en lugar de Data Ready simple: Aunque el enunciado solo exigía "al menos la interrupción Data Ready", se optó por FIFO watermark porque reduce la carga de interrupciones en un factor de 64×, mejorando la determinismo del sistema FreeRTOS y reduciendo la probabilidad de pérdida de muestras por latencias en el servicing de IRQs.

Fragmentación (chunking) de mensajes JSON: Para enviar 1024 muestras en modo continuo, se fragmentan en 16 chunks de 64 muestras cada uno. Esta decisión se tomó para evitar paquetes MQTT de >4KB que podrían saturar el buffer de red del módulo WiFi y causar timeouts. El protocolo de reconstrucción basado en campos `s` (secuencia) y `n` (total chunks) permite al receptor reensamblar la serie temporal completa.

Stack de 5120 bytes para `MQTT_Task`: Aunque parece excesivo comparado con las otras tareas (2048-4096 bytes), este valor se determinó tras debugging de un Hard Fault por stack overflow. La combinación del buffer de red de `coreMQTT` (1000 bytes) con las variables locales de contexto MQTT requiere este tamaño para garantizar margen de seguridad.

Compilación condicional con `NODE_ID`: En lugar de mantener dos proyectos separados (uno por nodo), se usa `#if NODE_ID == NODE_ID_ACCEL` para compilar selectivamente el código específico de cada nodo. Esto facilita el mantenimiento (un bug fix beneficia a ambos nodos) y garantiza que las tareas comunes (WiFi, MQTT, UART) son idénticas en ambas versiones.

2 DISEÑO DEL SISTEMA

2.1 Diseño de tareas

La arquitectura software del sistema se articula en torno a cinco tareas de FreeRTOS que operan de forma concurrente, comunicándose mediante colas de mensajes y notificaciones. La siguiente tabla resume las características de configuración de cada tarea:

Tabla 1: tareas

Tarea	Stack (bytes)	Prioridad	Colas que utiliza	Notificaciones que recibe
WifiTask	2048	osPriorityNormal	—	—
MQTT_Task	4096	osPriorityLow	qMqttTx (consume)	—
Accel_Task (NODE_ID=1)	4096	osPriorityNormal	qMqttTx (produce) qCmdRx (consume)	NOTE_BUTTON_IRQ NOTE_RTC_WAKEUP NOTE_ACCEL_FIFO
task_envRead (NODE_ID=2)	2048	osPriorityNormal	qMqttTx (produce) qCmdRx (consume)	NOTE_BUTTON_IRQ NOTE_RTC_WAKEUP
UartCfgTask	2048	osPriorityLow	qCmdRx (produce)	NOTE_UART_LINE

2.1.1 Asignación de Prioridades

La asignación de prioridades sigue criterios de tiempo real blando:

- WifiTask tiene prioridad Normal porque la conectividad es un requisito previo para el funcionamiento del sistema. Debe poder anticiparse a la ejecución de tareas de aplicación si necesita gestionar reconexiones urgentes.
- Tareas de sensores (Accel_Task, task_envRead) tienen prioridad Normal porque su función principal (adquisición de datos) debe ejecutarse con latencias predecibles al recibir notificaciones hardware (IRQs, RTC).
- MQTT_Task y UartCfgTask tienen prioridad Low porque son tareas de servicio que procesan colas a su ritmo natural sin restricciones de tiempo duro. Esta priorización evita que la transmisión de datos bloquee la adquisición de nuevas muestras.

2.1.2 Dimensionamiento de Stacks

Los tamaños de stack se determinaron mediante profiling empírico tras incidentes de stack overflow (Hard Fault):

- MQTT_Task (5120 bytes): Requiere el stack más grande debido al buffer de red NETWORK_BUFFER_SIZE (1000 bytes) de la librería coreMQTT, más las estructuras de contexto MQTTContext_t y las variables locales de funciones de serialización JSON. (4096 bytes): Dimensionado para el buffer de red NETWORK_BUFFER_SIZE (1000 bytes) de la librería coreMQTT más las estructuras de contexto. Gracias a la implementación con punteros en lugar de copia por valor de mensajes completos, se ha podido reducir desde los 5120 bytes iniciales manteniendo aproximadamente un 50 % de margen de seguridad.
- Accel_Task (4096 bytes): Necesita almacenar un buffer de payload JSON de 1024 bytes para construir los mensajes antes de encolarlos, más las variables locales de control de FIFO.
- Resto de tareas (2048 bytes): Stack estándar suficiente para sus operaciones, que no involucran buffers grandes ni recursión profunda.

2.2 Diseño de Comunicaciones

2.2.1 Comunicación Interna (Local al Nodo)

Las tareas dentro de un mismo nodo se comunican mediante colas de mensajes de FreeRTOS (message queues). Se utilizan dos colas:

Cola qMqttTx (Transmisión):

- Dirección: Tareas de sensor a MQTT_Task
- Capacidad: 8 punteros $\times$ 4 bytes = 32 bytes de memoria estática
- Gestión de memoria: Dinámica mediante pvPortMalloc() y vPortFree() del heap de FreeRTOS
- Tipo de mensaje: Punteros a estructuras MqttMsg_t (1060 bytes alocados dinámicamente)
- Semántica: Cola con productor múltiple (Accel_Task, task_envRead, log_printf()) y consumidor único (MQTT_Task)
- Timeout de escritura: 100ms (si la cola está llena o malloc falla, se descarta el mensaje)
- Patrón de uso: Los productores alocan memoria con pvPortMalloc(), encolan el puntero, y el consumidor libera con vPortFree() tras publicar

Cola qCmdRx (Recepción de Comandos):

- Dirección: UartCfgTask y MQTT callback a Tareas de sensor
- Capacidad: 5 elementos $\times$ sizeof(SystemCommandMsg_t)
- Tipo de mensaje: Estructura con campos command (enum), source (UART/MQTT), y parámetros específicos
- Semántica: Cola de productor múltiple y consumidor único (según NODE_ID)
- Despertar consumidor: Mediante osThreadFlagsSet() sobre la tarea correspondiente

Esta arquitectura de colas permite desacoplamiento temporal: las tareas no necesitan ejecutarse sincrónicamente; cada una opera a su ritmo natural, limitada únicamente por la capacidad de las colas.

2.2.2 Optimización de Memoria en Cola MQTT

La implementación de qMqttTx utiliza un patrón de gestión dinámica de memoria mediante punteros en lugar de copia por valor, reduciendo significativamente el footprint de memoria del sistema.

La implementación actual utiliza punteros a estructuras alocadas dinámicamente:

1. send_mqtt_msg() (Productor):
 - Aloca memoria del heap FreeRTOS: m = pvPortMalloc(sizeof(MqttMsg_t))
 - Verifica éxito de malloc y disponibilidad de MQTT (NET_MQTT_OK)
 - Construye el mensaje en la memoria alocada
 - Encola el puntero (4 bytes) en qMqttTx
 - Si el encolado falla, libera inmediatamente con vPortFree() para evitar leaks
2. MQTT_Task (Consumidor):
 - Desencola el puntero: osMessageQueueGet(qMqttTx, &msg_ptr, ...)
 - Utiliza el mensaje apuntado para publicación MQTT
 - Libera la memoria: vPortFree(msg_ptr)
 - Establece msg_ptr = NULL por seguridad

Ventajas de este enfoque:

- Reducción de memoria estática: 8480 → 32 bytes en la cola (99.6%)
- Reducción de stack del consumidor: 1060 → 4 bytes para la variable local
- Eliminación de copias: Solo se copia el puntero (4 bytes) en lugar de toda la estructura (1060 bytes) en cada operación de encolado/desencolado
- Flexibilidad: La memoria se aloca bajo demanda solo cuando hay mensajes

Trade-offs considerados:

- Fragmentación del heap: Con uso prolongado podría fragmentarse, aunque en la práctica no se ha observado degradación
- Complejidad: Requiere gestión manual de memoria y protección contra leaks
- Overhead de malloc/free: Despreciable comparado con latencias de red MQTT

2.2.3 Comunicación Externa (Red)

La comunicación entre nodos y el sistema de supervisión se realiza exclusivamente mediante MQTT sobre WiFi/TCP:

Patrón Publish/Subscribe:

- Publicación: Los nodos publican datos de sensores en topics jerárquicos (bridge/<tipo>/<id>). No hay comunicación directa nodo-a-nodo; toda la información fluye a través del broker central.
- Suscripción: Cada nodo se suscribe únicamente a su topic de control (bridge/cmd/<id>), garantizando que los comandos sean dirigidos y no broadcasts innecesarios.

Propiedades MQTT utilizadas:

- QoS 0 (At most once): No se requiere confirmación de entrega. Se prioriza latencia sobre garantías de entrega, asumiendo que la aplicación Python puede tolerar pérdidas ocasionales de paquetes (las siguientes muestras compensarán visualmente en las gráficas).
- Retain flag: false: Los mensajes no se almacenan en el broker. Solo interesa el flujo en tiempo real, no el estado histórico.
- Clean Session: true: Cada reconexión inicia sesión limpia sin suscripciones persistentes.

Esta elección de parámetros optimiza para baja latencia y bajo overhead, aceptando que en redes inestables puede haber pérdidas de datos puntuales.

2.2.4 Formato de mensajes

Todos los mensajes intercambiados vía MQTT utilizan JSON como formato de serialización. JSON se eligió por:

- Autodescriptivo (los campos llevan nombres explícitos)
- Directamente parseable en Python sin librerías externas (json.loads())
- Legible para debugging (se puede inspeccionar con clientes MQTT genéricos)
- Extensible (añadir campos no rompe compatibilidad hacia atrás)

Formato: Datos del acelerómetro

Debido al volumen de datos (aproximadamente 2KB en modo continuo), los mensajes del acelerómetro se fragmentan (chunking) en paquetes más pequeños. Cada chunk tiene el siguiente formato:

```
{
  "t0": 12345,
  "fs": 52,
```

```

"s": 0,
"n": 16,
"z": [1066, 1054, 1055, 1048, 1062, ..., 1051]
}

```

Tabla 2: campos del mensaje del acelerómetro

Campo	Tipo	Descripción	Rango/Valores
t0	entero	Timestamp de inicio de la captura completa (milisegundos desde boot)	0 - 2^{32}
fs	entero	Frecuencia de muestreo en Hz	52 (fijo)
s	entero	Número de secuencia de este chunk (0-indexed)	0 - (n-1)
n	entero	Total de chunks que componen la captura completa	1 (normal) / 16 (continuo)
z	array[entero]	Muestras del eje Z en mili g (mg)	Típicamente 950-1050 mg en reposo

En cuanto a las reglas de fragmentación, cada chunk contiene exactamente 64 muestras (el tamaño mínimo):

- Modo normal (64 muestras): Se envía 1 chunk con $s = 0$, $n = 1$
- Modo continuo (1024 muestras): Se envían 16 chunks con $s = 0 \dots 15$, $n=16$

En el receptor (código Python) se emplea un algoritmo de reconstrucción en el que se usa el campo t0 como identificador de sesión de captura, permitiendo al servidor distinguir chunks de diferentes capturas que podrían llegar entrelazados si el nodo realizara múltiples adquisiciones.

Formato: Datos ambientales

Los datos de temperatura y humedad se transmiten en un único mensaje JSON sin fragmentación:

```

{
  "id": 1,
  "msg_id": 42,
  "origen": 0,
  "temp": 235,
  "hum": 65
}

```

Tabla 3: campos de mensaje del nodo de medición ambiental

Campo	Tipo	Descripción	Rango/Valores
id	entero	Identificador del nodo (siempre 2 para nodo ambiental)	2
msg_id	entero	Contador secuencial de mensajes desde boot	0 - 2^{32}
origen	entero	Origen del disparo de lectura	0=RTC, 1=Botón, 2=Error
temp	entero	Temperatura en décimas de grado Celsius	Ej: 235 = 23.5°C
hum	entero	Humedad relativa en porcentaje	0 - 100

Se multiplica por 10 el valor leído por el sensor de temperatura para evitar punto flotante en JSON, reduciendo

el tamaño del payload y eliminando problemas de precisión en la deserialización. El receptor divide por 10 para recuperar el valor real. Por otra parte, el campo Origen permite al sistema de supervisión diferenciar entre lecturas periódicas automáticas (RTC), lecturas forzadas por el usuario (botón), o lecturas de emergencia tras detectar error en sensor (usado para debugging).

Formato: Comandos de control

Los comandos enviados a los nodos (topic bridge/cmd/<id>) son cadenas de texto planas, no JSON. Ejemplos:

```
READ
CONT ON
CONT OFF
WIFI miRed password123
RTC 2026 2 16 22 30 0
```

Esta decisión de diseño se tomó porque:

- Los comandos son suficientemente simples para no justificar la sobrecarga de JSON
- Facilita el debugging (se pueden enviar manualmente desde cualquier cliente MQTT)
- Permite reutilizar el mismo parser para UART y MQTT sin conversión JSON a texto

El parsing se realiza mediante `sscanf()` con patrones específicos por comando verificados en `Cmd_ParseLine()`

2.3 Topics MQTT del sistema

El sistema organiza la comunicación MQTT en tres familias de topics, parametrizadas por el identificador del nodo (<id>). Esto permite reutilizar la misma estructura en múltiples nodos y escalar la arquitectura sin cambios conceptuales.

Control y configuración

- `bridge/cmd/<id>`: canal de órdenes dirigido a un nodo concreto. Se utiliza para modificar el comportamiento en tiempo de ejecución (sin reflashear), tanto desde un cliente externo como desde otro nodo.

Publicación de datos

- `bridge/accel/1`: datos de acelerómetro del Nodo 1 (bloques de muestras en JSON, típicamente el eje Z y metadatos de captura).
- `bridge/env/2`: datos ambientales del Nodo 2 (temperatura/humedad en JSON, con identificador de mensaje y motivo de envío).

Logs y depuración

- `bridge/log/<id>`: canal de trazas del nodo. Permite supervisión remota (por ejemplo, estado de conexión WiFi/MQTT, recepción de comandos, cambios de modo), complementando la salida local por UART/SWV.

Comunicación entre nodos

- El Nodo ENV puede actuar como emisor de control hacia el Nodo ACC publicando en `bridge/cmd/1` (por ejemplo, para activar/desactivar modos). Esto demuestra un esquema de control distribuido basado en topics sin enlaces punto a punto.

3 IMPLEMENTACIÓN

3.1 Gestión del WiFi

La conectividad WiFi constituye la capa física de comunicaciones del sistema distribuido de monitorización. Su implementación se centra en garantizar la robustez del enlace inalámbrico mediante mecanismos de reconexión automática y configuración dinámica de credenciales, permitiendo que el sistema opere de forma autónoma sin intervención manual incluso ante caídas temporales de la red.

El módulo WiFi (ISM43362) se gestiona a través de la tarea WifiTask, cuyo objetivo es establecer y mantener la asociación con el punto de acceso configurado. La arquitectura implementada separa claramente las fases de inicialización (ejecutada una única vez al arranque) del bucle de conexión/reconexión (ejecutado continuamente). El estado de la conectividad se expone al resto del sistema mediante el flag global `WIFI_IS_CONNECTED`, que actúa como condición de arranque para las capas superiores (MQTT).

Una característica destacable del diseño es la capacidad de reconfigurar las credenciales WiFi (SSID y contraseña) en tiempo de ejecución mediante comandos UART o MQTT, almacenándolas en variables globales (`g_wifi_ssid`, `g_wifi_pass`). Esto elimina la necesidad de recompilar y reprogramar el firmware para cambiar de red, facilitando el despliegue del sistema en diferentes ubicaciones sin modificación de código.

3.1.1 Tarea de gestión WiFi (WifiTask)

La tarea WifiTask es responsable de establecer y mantener la conectividad con la red inalámbrica. Su implementación se basa en un bucle de reconexión infinito que detecta caídas de enlace y reintenta la conexión de forma automatizada, sin intervención del usuario ni de otras tareas del sistema.

3.1.1.1 Inicialización del Módulo WiFi

El arranque de la tarea comienza con la llamada a funciones de inicialización del driver ES-WIFI (módulo ISM43362), concretamente `WIFI_Init()` y `WIFI_GetMAC_Address()`. Estas operaciones se ejecutan una única vez al inicio y sirven para:

- Configurar los parámetros SPI del módulo WiFi.
- Realizar un reset físico del chip mediante GPIO.
- Verificar la comunicación con el módulo obteniendo su dirección MAC.

Si la inicialización falla, la tarea entra en el handler de errores (`Error_Handler()`), deteniendo la ejecución del firmware. Esta decisión de diseño refleja que la conectividad WiFi es un requisito crítico del sistema: sin red, los nodos no pueden cumplir su función de monitorización distribuida.

3.1.1.2 Bucle de Conexión y Reconexión

Una vez inicializado el módulo, WifiTask entra en su bucle principal infinito, donde ejecuta repetidamente `wifi_connect()`. Esta función implementa una máquina de estados implícita con tres fases:

1. **Intento de asociación:** Se invoca `WIFI_Connect(g_wifi_ssid, g_wifi_pass, WIFISEcurity)`, que utiliza las credenciales almacenadas en variables globales. Estas variables pueden ser modificadas dinámicamente mediante comandos UART/MQTT (comando `WIFI <ssid> <pass>`), permitiendo reconfigurar la red sin necesidad de recompilar ni reprogramar el firmware.
2. **Obtención de dirección IP:** Si la asociación tiene éxito, se llama a `WIFI_GetIP_Address(IP_Addr)` para confirmar que el nodo ha recibido una IP válida del servidor DHCP. Este paso valida que el enlace es funcional a nivel de red (capa 3 OSI).
3. **Activación de flag global:** Solo si ambas operaciones anteriores son exitosas, se activa el flag `WIFI_IS_CONNECTED = 1`. Este flag es monitoreado de forma asíncrona por `MQTT_Task`,

desencadenando el inicio de las operaciones de la capa aplicación.

En caso de fallo en cualquiera de los pasos anteriores, la función devuelve un código de error (-1) y WifiTask introduce un delay de 2 segundos antes de reintentar, evitando saturar el medio inalámbrico con intentos de conexión continuos. Este comportamiento de backoff es esencial para cumplir con las buenas prácticas de diseño de redes WiFi.

3.1.1.3 Mantenimiento de la Conexión

Aunque el código actual no implementa un mecanismo explícito de watchdog para detectar caídas del enlace WiFi una vez establecido, la arquitectura facilita su incorporación futura. El driver ES-WIFI proporciona funciones para verificar el estado de la conexión (WIFI_GetConnectionStatus()), que podría invocarse periódicamente dentro del bucle de WifiTask para detectar desconexiones silenciosas y forzar la reconexión.

La ausencia de este mecanismo en la implementación actual se compensa parcialmente con la lógica de MQTT_Task: si el broker MQTT deja de responder a los mensajes KeepAlive, se interpreta como una pérdida del enlace de red y se reinicia todo el ciclo de conexión (incluyendo TCP + MQTT handshake).

3.2 MQTT

El protocolo MQTT constituye la capa de aplicación del sistema de comunicaciones, operando sobre la infraestructura WiFi/TCP establecida por WifiTask. Su implementación se centra en dos flujos de datos bidireccionales: (1) publicación de lecturas de sensores hacia el broker central, y (2) recepción de comandos de control remoto desde plataformas de supervisión.

La tarea MQTT_TaskFun gestiona el ciclo de vida completo del cliente MQTT, desde el handshake inicial con el broker hasta el mantenimiento de la sesión mediante mensajes KeepAlive. La arquitectura implementa un patrón productor-consumidor donde las tareas de sensores (Accel_Task, task_envRead) actúan como productores que depositan sus datos en una cola unificada (qMqttTx), mientras que MQTT_Task consume estos mensajes y los transmite de forma serializada al broker. Este desacoplamiento permite que la adquisición de datos opere a frecuencias deterministas (52 Hz para acelerómetro) sin verse afectada por las latencias variables de la red.

El flag global NET_MQTT_OK sincroniza el estado de la sesión MQTT con el resto del sistema. Solo cuando este flag está activo, las funciones de publicación (send_mqtt_msg(), log_printf()) encolan mensajes en qMqttTx, evitando pérdidas de datos si MQTT no está listo. Si la sesión MQTT falla (por timeout de broker, pérdida de WiFi, etc.), el flag se desactiva automáticamente y MQTT_Task reinicia todo el proceso de conexión desde cero, garantizando que el sistema nunca opere en estados inconsistentes.

Un aspecto destacable es la integración con el sistema de comandos: MQTT_Task se suscribe dinámicamente al topic bridge/cmd/<NODE_ID>, registrando un callback que parsea los comandos recibidos y los distribuye a las tareas de aplicación mediante la misma infraestructura usada para comandos UART. Esto proporciona transparencia de canal: la lógica de negocio no distingue si un comando llegó por consola local o por red remota.

3.3 Tarea de gestion de MQTT (MQTT_TaskFun)

La tarea MQTT_TaskFun implementa la capa de aplicación del sistema de comunicaciones, gestionando tanto la publicación de datos de sensores como la recepción de comandos de control remoto. Su diseño se estructura en cinco fases claramente diferenciadas, ejecutadas secuencialmente en un bucle infinito.

3.3.1 Fase 1: Espera de Conectividad WiFi

La tarea comienza ejecutando un bucle de espera activa (while (WIFI_IS_CONNECTED == 0)) con delays de 500ms entre iteraciones. Esta lógica de sincronización garantiza que no se intente establecer una conexión MQTT sin que exista un enlace WiFi subyacente, evitando fallos en prvConnectToServer() por falta de conectividad de red.

El uso de polling con delays introducidos manualmente (en lugar de notificaciones desde WifiTask) simplifica la implementación y es aceptable dado que esta fase solo se ejecuta en el arranque del sistema o tras una desconexión completa. En futuras iteraciones, podría optimizarse mediante el uso de thread flags para despertar MQTT_Task inmediatamente cuando WIFI_IS_CONNECTED cambie a 1.

3.3.2 Fase 2: Conexión TCP al Broker

Una vez verificada la disponibilidad de WiFi, se invoca `prvConnectToServer(&xNetworkContext)`, función proporcionada por la librería `coreMQTT` que encapsula la creación de un socket TCP y el handshake con el broker. Los parámetros del broker (dirección IP/dominio y puerto) se definen en `mqtt_priv_config.h`.

La función devuelve un `TransportStatus_t` que indica éxito (`PLAINTEXT_TRANSPORT_SUCCESS`) o fallo. En caso de error, `MQTT_Task` introduce un delay de 2 segundos y reinicia el ciclo completo (volviendo a la fase 1). Esta estrategia de reinicio total garantiza que cualquier estado inconsistente en las estructuras internas de la librería quede descartado en cada intento.

Es importante destacar que `prvConnectToServer()` ya incluye internamente un delay de 10 segundos en caso de fallo de conexión TCP, por lo que el delay adicional de 2 segundos en nuestro código actúa como un segundo nivel de backoff.

3.3.3 Fase 3: Handshake MQTT y Suscripción

Si la conexión TCP tiene éxito, se procede al handshake de la capa MQTT mediante `prvCreateMQTTConnectionWithBroker(&xMQTTContext, &xNetworkContext)`. Esta función:

- Envía el paquete MQTT CONNECT con el Client ID definido en configuración.
- Espera el paquete CONNACK del broker.
- Inicializa las estructuras de estado del contexto MQTT.

La implementación original de esta función no devuelve códigos de error; si el handshake falla, ejecuta internamente `configASSERT()`, lo que provoca un reset completo del microcontrolador. Este comportamiento, aunque drástico, garantiza que el sistema nunca opere en un estado inconsistente de "medio conectado".

Una vez establecida la sesión MQTT, la tarea se suscribe dinámicamente al topic de control específico de su nodo:

```
char subTopic[64];
snprintf(subTopic, sizeof(subTopic), "%s%d", TOPIC_SUB_CMD_PREFIX, NODE_ID);
prvMQTTSubscribeToTopic(&xMQTTContext, subTopic);
```

Donde `TOPIC_SUB_CMD_PREFIX` vale "bridge/cmd/" y `NODE_ID` es 1 o 2 según el tipo de nodo (acelerómetro o environment). Esto genera topics del tipo `bridge/cmd/1` o `bridge/cmd/2`, permitiendo control individualizado de cada nodo sin necesidad de cambiar código entre compilaciones.

Finalmente, se activa el flag global `NET_MQTT_OK = 1`, indicando al resto del sistema que la publicación MQTT está operativa. Este flag es verificado por `send_mqtt_msg()` antes de encolar mensajes, evitando pérdidas de datos si MQTT no está listo.

3.3.4 Fase 4: Bucle de Publicación y KeepAlive

El corazón funcional de `MQTT_Task` es el bucle de transmisión, que ejecuta dos operaciones críticas en cada iteración:

1. Publicación de mensajes: Se consulta la cola `qMqttTxHandle` con un timeout de 100ms. Si hay mensajes pendientes, se extraen y se publican usando la API de `coreMQTT`. La estructura `MqttMsg_t` (definida en `common_def.h`) encapsula tanto el topic de destino como el payload JSON, permitiendo que las tareas productoras (sensores) especifiquen dinámicamente el canal de publicación sin conocer detalles de MQTT.
2. Mantenimiento de sesión (KeepAlive): La función `MQTT_ProcessLoop()` es invocada en cada iteración (cada aproximadamente 100ms) y realiza:
 - Envío de paquetes MQTT PINGREQ si ha transcurrido el intervalo de KeepAlive.
 - Recepción y procesamiento de paquetes entrantes del broker (PINGRESP, PUBLISH con

comandos, etc.).

- Actualización de timers internos del protocolo MQTT.

Si esta función detecta un error (timeout en PINGRESP, socket cerrado, etc.), devuelve un código distinto de MQTTSuccess, lo que provoca la salida del bucle de publicación y la transición a la fase de limpieza.

3.3.5 Fase 5: Limpieza y Reinicio del Ciclo

Al detectar un error en el KeepAlive o en la publicación, MQTT_Task ejecuta las siguientes acciones:

- Desactiva el flag NET_MQTT_OK = 0, bloqueando nuevas inserciones en la cola de mensajes.
- Introduce un delay de 1 segundo para permitir que el stack TCP/IP libere recursos.
- Reinicia el bucle infinito, volviendo a la fase 1 (espera de WiFi).

Este reinicio completo del ciclo garantiza que todos los contextos (TCP + MQTT) se creen desde cero, descartando cualquier estado corrupto que pudiera haberse generado durante el fallo de conexión.

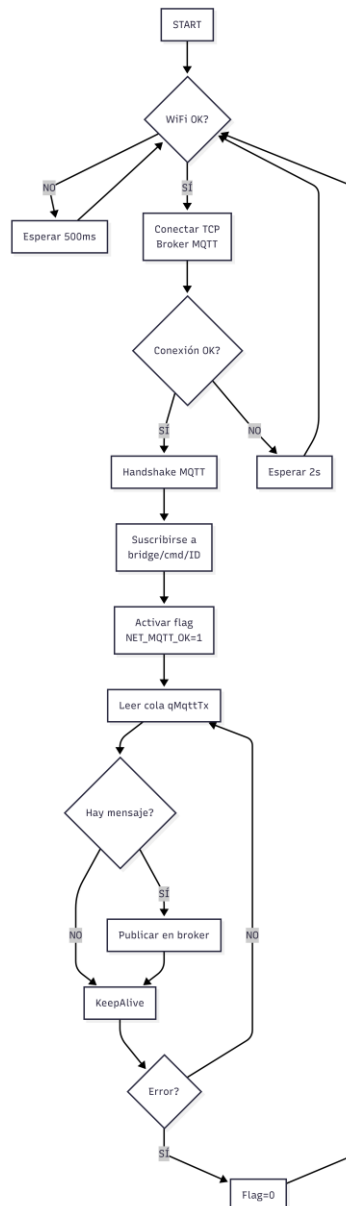


Figura 1: Diagrama de flujo de MQTT_Task

3.4 Configuración por UART y MQTT. Parseo de comandos

Se ha desarrollado un sistema de configuración unificado que permite controlar el firmware por dos vías: UART (local) y MQTT (remota). En ambos casos, las órdenes se expresan como líneas de texto con el mismo formato (por ejemplo, `READ, CONT ON, WIFI <ssid> <pass>, RTC . . .`). El objetivo es que el canal de entrada sea transparente: independientemente de si el comando llega por serie o por MQTT, el procesamiento interno es idéntico y desemboca en una estructura común (`SystemCommandMsg_t`) que la aplicación ejecuta de forma segura desde sus tareas.

A nivel interno, el flujo se organiza en tres etapas: (1) recepción y encapsulado, (2) parseo, y (3) distribución a la tarea correspondiente. La recepción por UART se apoya en interrupciones para no bloquear y detectar fin de línea; la recepción por MQTT se produce a través del callback de la librería tras la suscripción al topic de control del nodo. En ambos casos, el texto se valida y se traduce a un mensaje estructurado mediante `Cmd_ParseLine()`. Finalmente, el comando se inserta en una cola de mensajes (`qCmdRx`) y se despierta la tarea de aplicación adecuada mediante flags de FreeRTOS, garantizando que la ejecución del comando se realiza en contexto de tarea (no dentro de ISR ni callback de red), evitando latencias y efectos no deterministas.

3.4.1 Tarea de configuración por UART (`UartCfgTask_Func`)

La tarea `UartCfgTask_Func` es el componente encargado de convertir la UART en una interfaz de consola fiable y no intrusiva. Su función principal es recibir líneas completas, parsearlas y distribuirlas al sistema de comandos. Para conseguirlo, el firmware separa claramente dos responsabilidades:

- **En interrupción (`HAL_UART_RxCpltCallback`):** solo se realiza trabajo mínimo y determinista. Se acumulan caracteres en un buffer (`uart_line`) y, cuando se detecta `\r` o `\n`, se copia la línea a un buffer “listo” (`uart_line_ready`) y se marca como disponible (`uart_line_has_ready`). En ese momento se despierta la tarea mediante `osThreadFlagsSet(UartCfgTaskHandle, NOTE_UART_LINE)`. Esta elección evita parsear en ISR y reduce el riesgo de bloquear el sistema.
- **En tarea (`UartCfgTask_Func`):** la tarea permanece bloqueada con `osThreadFlagsWait(NOTE_UART_LINE, ...)` hasta que existe una línea completa. Cuando se activa el flag, la tarea lee `uart_line_ready`, llama a `Cmd_ParseLine(...)` para validar y traducir el texto a `SystemCommandMsg_t`, y si el comando es válido lo entrega al sistema mediante `DispatchCommand(&cmd)`. Si el comando no tiene formato correcto, se genera un log informativo y se descarta.

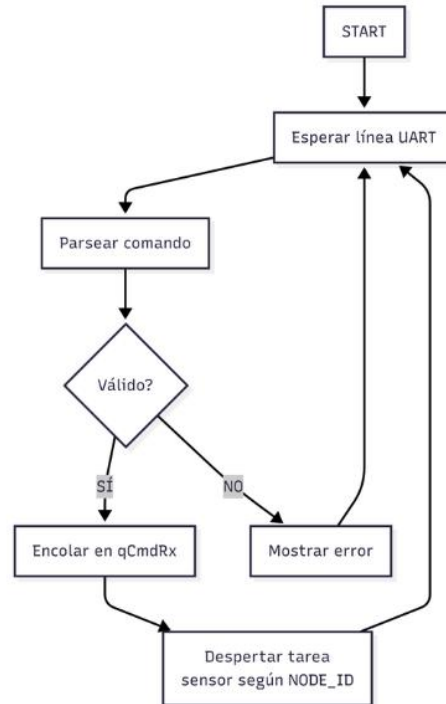


Figura 2. Diagrama de flujo UartCfgTask_Fun

3.4.2 Recepción remota por MQTT y unificación del parseo

La configuración remota se implementa suscribiendo cada nodo a su topic de control `bridge/cmd/<NODE_ID>`. Cuando llega un mensaje, la función `OnMqttControlMessage(const char *topic, const char *payload)` toma el payload (el texto del comando) y ejecuta exactamente el mismo flujo que en UART: se llama a `Cmd_ParseLine(payload, CMD_SRC_MQTT, &cmd)` y, si el parseo es correcto, se entrega con `DispatchCommand(&cmd)`.

La consecuencia práctica es que el sistema tiene un único “idioma de comandos” y un único parser, lo que reduce errores y evita duplicar lógica. Además, esta unificación permite que otros nodos (por ejemplo, el nodo ENV) envíen órdenes al nodo ACC publicando en `bridge/cmd/1`, lo que habilita control distribuido sin introducir protocolos adicionales.

3.4.3 Distribución y ejecución en tareas de aplicación

Una vez que un comando ha sido recibido (por UART o por MQTT) y validado por el parser, se entrega al sistema mediante un mecanismo común basado en cola + señalización por eventos. En concreto, `DispatchCommand(...)` introduce el comando ya estructurado (`SystemCommandMsg_t`) en la cola `qCmdRx` y, a continuación, despierta la tarea de aplicación que corresponda al nodo (tarea ACC o tarea ENV) utilizando flags (`NOTE_CMD_RX`).

Este enfoque garantiza que la ejecución del comando se realiza siempre en contexto de tarea, y no dentro de una ISR ni dentro de un callback de red. Esto es importante porque muchas acciones asociadas a comandos pueden implicar operaciones relativamente costosas o sensibles a temporización (por ejemplo, reconfigurar parámetros, rearme de RTC, forzar una lectura, o iniciar un proceso de reconexión WiFi). Al ejecutarse en la tarea principal del nodo, dichas acciones quedan coordinadas por el planificador y se evita introducir bloqueos, latencias no deterministas o efectos secundarios sobre el resto del sistema.

Además, al emplear una cola intermedia, el sistema puede absorber ráfagas de comandos sin perder mensajes: los comandos quedan almacenados y la tarea los procesa en orden, vaciando la cola en cada activación. Este patrón también simplifica la extensión a nuevos comandos o a nuevos roles de nodo, ya que la recepción y el

parseo permanecen invariables y únicamente se amplía la lógica de gestión dentro de la tarea de aplicación correspondiente.

3.5 NODO 1: Acelerómetro y FIFO

El nodo acelerómetro (identificado como **Nodo 1**, `NODE_ID = 1`) constituye el subsistema encargado de medir vibraciones o aceleraciones mediante el sensor **LSM6DSL** integrado en la placa. Su objetivo principal es transformar esas lecturas en información útil y publicarla a través de **MQTT**, de forma que un cliente externo (PC, servidor o herramienta de monitorización) pueda recibir los datos de manera prácticamente inmediata.

Además de la adquisición, el nodo incorpora una capa de **control remoto**, lo que permite modificar su comportamiento dinámicamente sin necesidad de recompilar ni reprogramar la placa. En concreto, es posible forzar capturas puntuales, activar o desactivar el modo continuo, reconfigurar credenciales WiFi o ajustar el RTC. Esta filosofía convierte al nodo en un componente flexible dentro de la arquitectura IoT: no se limita a medir y enviar, sino que puede adaptarse a órdenes externas en tiempo de ejecución.

Un aspecto clave de la implementación es la utilización de la **FIFO interna** del LSM6DSL. En lugar de leer muestras una a una mediante sondeo (polling), el sensor acumula automáticamente muestras en su memoria FIFO y el microcontrolador las recupera en **bloques** cuando el sensor indica que hay suficiente información disponible. Este planteamiento reduce el número de accesos al bus, disminuye la carga de CPU y mejora la robustez temporal del muestreo. La sincronización entre hardware y software se realiza mediante el mecanismo de **watermark**, que provoca una interrupción cuando el nivel de FIFO alcanza un umbral prefijado.

3.5.1 Tareas y funciones

3.5.1.1 Accel_Task_Func (tarea de adquisición del acelerómetro)

La tarea `Accel_Task_Func` es la encargada de orquestar la captura de aceleración del Nodo 1 utilizando el sensor LSM6DSL en modo FIFO. Su comportamiento está basado en eventos y control remoto, por lo que no ejecuta lecturas constantes por “polling”, sino que permanece bloqueada hasta recibir una señal de trabajo.

En cada ciclo, la tarea:

- **Espera eventos** mediante `osThreadFlagsWait()`, que pueden provenir de:
 - **RTC** (`NOTE_RTC_WAKEUP`): captura periódica.
 - **Botón** (`NOTE_BUTTON_IRQ`): captura manual.
 - **Comandos** (`NOTE_CMD_RX`): cambios de modo (`CONT ON/OFF`), lectura forzada (`READ`), configuración WiFi o RTC.
- **Decide el modo de captura:**
 - *Normal*: adquiere un bloque corto (p. ej. 64 muestras).
 - *Continuo*: adquiere un bloque largo (p. ej. 1024 muestras) fragmentado en paquetes.
- **Configura y resetea FIFO** antes de capturar para garantizar que el bloque enviado sea coherente y sin datos residuales.
- **Espera interrupciones del watermark** (`INT1` → `EXTI`) para saber cuándo hay datos suficientes en FIFO.
- **Lee muestras y forma bloques**, construyendo JSON con las muestras de Z y enviándolo a MQTT mediante cola de mensajes.

Este diseño permite capturar datos en bloques con bajo consumo de CPU y una sincronización fiable entre hardware (interrupciones FIFO) y software (tarea).

3.5.1.2 LSM6DSL_FifoReset(void)

Esta función vacía la FIFO de forma fiable. En el LSM6DSL no existe una orden directa de “clear FIFO”, por lo que se utiliza la técnica habitual basada en cambiar temporalmente el modo de funcionamiento: se coloca la FIFO en modo bypass (deshabilitada), lo que descarta el contenido acumulado, y a continuación se vuelve a activar el modo FIFO para dejar el buffer listo para la siguiente captura.

Esta operación es especialmente relevante antes de iniciar una adquisición, ya que evita que se mezclen muestras antiguas con nuevas, por ejemplo tras interrupciones previas o cambios de modo (normal ↔ continuo).

3.5.1.3 LSM6DSL_FifoConfig(uint16_t samples_xyz)

La función `LSM6DSL_FifoConfig()` configura el sensor para que el acelerómetro escriba datos en la FIFO y se genere el evento de watermark cuando exista suficiente información acumulada. En primer lugar calcula el umbral en words de 16 bits (unidad interna con la que el sensor expresa el nivel FIFO). Como cada muestra completa XYZ ocupa 3 words (X, Y, Z), el watermark se calcula como `samples_xyz * 3`.

A continuación habilita en `CTRL3_C` dos mecanismos fundamentales para una lectura consistente: BDU y autoincremento. BDU (Block Data Update) impide que el sensor actualice registros en mitad de una lectura, evitando lecturas “mezcladas” o incoherentes. El autoincremento (`IF_INC`) permite leer registros consecutivos en una sola operación, algo imprescindible cuando se realizan lecturas múltiples de datos (por ejemplo, 6 bytes para XYZ).

La configuración fuerza además los parámetros de muestreo esperados en esta práctica: el acelerómetro se ajusta a 52 Hz y ± 2 g mediante `CTRL1_XL`, asegurando que la FIFO recibe datos con ese ritmo y escala. Posteriormente se habilita el flujo del acelerómetro dentro de la FIFO (registro `FIFO_CTRL3`) y se programa el watermark en `FIFO_CTRL1` y `FIFO_CTRL2` (parte baja y parte alta del umbral).

Finalmente se configura `FIFO_CTRL5` para fijar el ODR de la FIFO y el modo FIFO, y se habilita la interrupción de watermark en el pin INT1 (registro `INT1_CTRL`). Gracias a esto, cuando se alcanza el umbral, el sensor activa INT1 y el microcontrolador recibe un evento EXTI que despierta la tarea del acelerómetro.

En conjunto, esta función convierte al LSM6DSL en un productor de datos por bloques sincronizado por interrupciones, reduciendo la necesidad de polling y mejorando la estabilidad temporal del muestreo.

3.5.1.4 LSM6DSL_FifoGetLevelWords(void)

Devuelve el nivel actual de ocupación de FIFO en *words*. Para ello lee `FIFO_STATUS1` (parte baja) y `FIFO_STATUS2` (parte alta, 4 bits), combinándolos en un entero de 12 bits.

Este valor permite al sistema:

- Saber si realmente hay datos suficientes aunque se haya perdido una interrupción (fallback).
- Consumir exactamente el contenido disponible sin leer de más.
- Comprobar si el watermark se alcanzó o si aún falta información.

3.5.1.5 LSM6DSL_FifoReadXYZRaw(int16_t *x, int16_t *y, int16_t *z)

Lee una muestra completa XYZ desde la FIFO y la devuelve en formato RAW (enteros de 16 bits). Internamente:

- Hace una lectura múltiple de 6 bytes desde FIFO_DATA_OUT_L.
- Reconstruye cada eje como `low_byte + (high_byte << 8)`.

Esta función es la “unidad básica” de consumo de FIFO: cada llamada extrae exactamente una muestra (X,Y,Z). En la aplicación, normalmente se usa para leer repetidamente mientras el nivel FIFO indique que quedan datos.

3.5.1.6 `send_mqtt_msg(const char *topic, const char *payload)`

Encapsula el envío de mensajes MQTT. En lugar de publicar directamente, construye un `MqttMsg_t` y lo introduce en la cola `qMqttTx`. Esto desacopla la tarea de adquisición del hilo MQTT, evitando bloqueos durante la captura.

3.5.1.7 `build_payload_block(char *dst, size_t dst_sz, ...)`

Construye el payload en formato JSON. Su objetivo es generar un mensaje compacto con metadatos (timestamp inicial, frecuencia, índice de fragmento, número total de fragmentos) y el vector de muestras (Z). Permite fragmentar capturas largas en bloques homogéneos.

3.5.2 Funcionamiento

El Nodo 1 opera como un subsistema de adquisición orientado a eventos: no está leyendo el acelerómetro de forma constante, sino que permanece a la espera y solo inicia una captura cuando ocurre un disparador. Al arrancar, la tarea inicializa el sensor con `BSP_ACCELERO_Init()` y programa una alarma RTC mediante `program_alarm_RTC()`, que actúa como mecanismo de adquisición periódica. La tarea queda entonces bloqueada en `osThreadFlagsWait()` esperando tres posibles estímulos: disparo del RTC (`NOTE_RTC_WAKEUP`), pulsación del botón (`NOTE_BUTTON_IRQ`) o llegada de un comando (`NOTE_CMD_RX`).

Los comandos pueden llegar por UART o por MQTT, pero siguen el mismo flujo interno: el texto se parsea con `Cmd_ParseLine()`, se empaqueta en un `SystemCommandMsg_t` y se introduce en `qCmdRx`. Posteriormente se despierta la tarea del acelerómetro, que vacía la cola y aplica las acciones necesarias. Esto permite modificar el comportamiento sin recompilar: activar modo continuo (`CONT ON`), volver al modo normal (`CONT OFF`) o forzar una lectura inmediata (`READ`). Aunque este sea el nodo del acelerómetro, también acepta configuración global como cambio de WiFi o ajuste del RTC.

Cuando se inicia una captura (por RTC, botón o `READ`), la tarea selecciona el tamaño objetivo según el modo. En modo normal se adquiere un bloque corto (por ejemplo 64 muestras) y en modo continuo se adquiere un bloque largo (por ejemplo 1024 muestras), que posteriormente se fragmenta en varios mensajes MQTT. Tras la captura periódica, la tarea **rearma la alarma RTC** para mantener el muestreo periódico.

Antes de comenzar la adquisición, el nodo prepara el sensor para utilizar la FIFO como buffer de muestreo. Para ello limpia el contenido previo con `LSM6DSL_FifoReset()` y configura el funcionamiento de la FIFO mediante `LSM6DSL_FifoConfig(watermark_samples)`, fijando la configuración de muestreo (**52 Hz y ±2 g**), activando la escritura del acelerómetro dentro de la FIFO y programando el umbral de watermark. Un punto esencial es que se habilita el evento de watermark en el pin **INT1** del sensor, de manera que el propio acelerómetro genera un pulso cuando se alcanza el umbral de datos. Además, se limpian flags antiguos (`NOTE_ACCEL_FIFO`) para evitar que un evento residual provoque lecturas fuera de momento.

A partir de ese punto, el LSM6DSL almacena muestras en su FIFO interna de forma autónoma. Cuando se alcanza el watermark, el sensor activa INT1 y se dispara una interrupción EXTI en el microcontrolador. En el callback EXTI, el evento hardware se traduce a un evento del sistema operativo mediante

`osThreadFlagsSet(Accel_TaskHandle, NOTE_ACCEL_FIFO)`, despertando la tarea del acelerómetro. Por robustez, la tarea no depende únicamente del flanco de interrupción: consulta también el nivel real del FIFO con `LSM6DSL_FifoGetLevelWords()`. Si se perdiera un flanco pero la FIFO ya contiene los datos necesarios, la captura puede continuar sin bloqueo.

La lectura se realiza consumiendo la FIFO muestra a muestra con `LSM6DSL_FifoReadXYZRaw()`, extrayendo cada paquete XYZ. En la aplicación se utiliza principalmente el eje Z para representar vibración vertical. Las muestras se agrupan en bloques (`ACC_BLOCK_SAMPLES`) y, al completarse cada bloque, se construye un JSON compacto con `build_payload_block()`, incluyendo metadatos (instante de referencia, frecuencia y fragmentación) y el array `z[]`. El envío no se realiza directamente: se encola mediante `send_mqtt_msg()` en `qMqttTx`, y es la tarea MQTT la que publica en el topic correspondiente (`bridge/accel/1`).

El proceso continúa hasta alcanzar el número total de muestras objetivo (64 en modo normal o 1024 en modo continuo). Al finalizar, se resetea nuevamente la FIFO para dejar el sensor en estado limpio, se registran logs de fin de captura y la tarea vuelve a estado de espera. Con este diseño, el nodo combina adquisición eficiente (FIFO + watermark + EXTI), control remoto (UART/MQTT), publicación estructurada (JSON por MQTT) y tolerancia ante fallos comunes en sistemas embebidos (pérdida de interrupciones o variaciones temporales), manteniendo además una integración natural con el resto de la arquitectura IoT.

A continuación, se muestra un diagrama de flujo de la tarea

- **Espera eventos** mediante `osThreadFlagsWait()`, que pueden provenir de:
 - **RTC** (`NOTE_RTC_WAKEUP`): captura periódica.
 - **Botón** (`NOTE_BUTTON_IRQ`): captura manual.
 - **Comandos** (`NOTE_CMD_RX`): cambios de modo (`CONT ON/OFF`), lectura forzada (`READ`), configuración WiFi o RTC.
- **Decide el modo de captura:**
 - *Normal*: adquiere un bloque corto (p. ej. 64 muestras).
 - *Continuo*: adquiere un bloque largo (p. ej. 1024 muestras) fragmentado en paquetes.
- **Configura y resetea FIFO** antes de capturar para garantizar que el bloque enviado sea coherente y sin datos residuales.
- **Espera interrupciones del watermark** (`INT1` → `EXTI`) para saber cuándo hay datos suficientes en FIFO.
- **Lee muestras y forma bloques**, construyendo JSON con las muestras de Z y enviándolo a MQTT mediante cola de mensajes.

3.6 NODO 2: Temperatura y humedad

El nodo de sensor de entorno (identificado como nodo 2 en `NODE_ID = 2`) es el nodo encargado de realizar las lecturas de temperatura y humedad relativa con el sensor `hts221`, integrado en la placa. El objetivo principal de este nodo es tomar las lecturas y convertirlas en información útil para el usuario, para que un cliente externo pueda visualizarlas de forma fácil y sencilla, al igual que con el nodo acelerómetro.

Este nodo incorpora una capa de control remoto, al igual que el nodo acelerómetro, capaz de cambiar su comportamiento, aunque no se apreciarán cambios significativos ante todas las instrucciones que se le envíen.

3.6.1 Tareas y funciones

3.6.1.1 Env_read_task_func

Esta tarea es la encargada de coordinar las lecturas de temperatura y humedad, y activar la alarma si fuera necesario. La función despierta cada media hora con una alarma del RTC, lo que garantiza que la función duerma durante el período de inactividad, eliminando parte del consumo. En cada ciclo, la tarea hace lo siguiente:

- **Espera eventos** mediante `osThreadFlagsWait()`, que pueden provenir de:
 - **RTC** (`NOTE_RTC_WAKEUP`): captura periódica.
 - **Botón** (`NOTE_BUTTON_IRQ`): captura manual.
 - **Comandos** (`NOTE_CMD_RX`): cambios de modo (`CONT ON/OFF`), lectura forzada (`READ`), configuración WiFi o RTC.
- **Procesa los datos:**
 - Formatea la temperatura como un número con un decimal, y la humedad como un número sin decimales.
 - Convierte ambos datos a integer.
- **Activa la alarma** si la temperatura sube de 36°C (en el enunciado no se especifica un umbral)
- **Lee muestras y forma bloques**, construyendo un JSON con las medidas tomadas, para que sean subidas a la cola y enviadas al proceso encargado de enviar por MQTT.

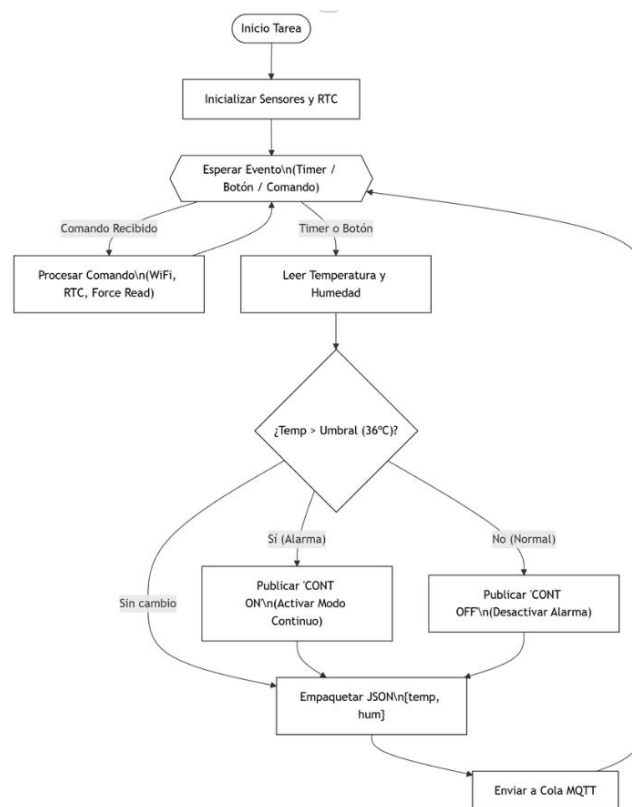


Figura 3. Diagrama de flujo envRead_Task_Func.

El JSON tiene los siguientes campos:

- **Identificador del nodo:** Identifica el nodo que envía la trama.
- **Timestamp:** Marca de tiempo, para poder hacer análisis temporales.

- **Identificador del mensaje:** Identifica el mensaje, para saber si se ha perdido alguno.
- **Motivo:** 0 si el mensaje se ha generado por una alarma, y 1 si se ha generado por comando o por pulsación del botón.
- **Temperatura**
- **Humedad**

3.6.1.2 Program_RTC()

Esta función es la encargada de programar la alarma que despierta al proceso `env_read_task_func`. Es llamada para configurar la alarma por primera vez, y se le llama cada vez que suena la alarma. El funcionamiento es simple: mira que hora marca el RTC, y calcula el momento en el que la alarma debe sonar. Acto seguido, configura el RTC para que genere una interrupción en el momento calculado.

```
{"id":1,"ts":"2000-01-01 00:34:00","msg_id":39,"origen":"0","temp":246,"hum":53}
```

Figura 4. JSON generado.

3.7 Logs por UART y MQT

Se ha implementado un sistema de trazas orientado a depuración y supervisión, con doble salida: **local** mediante UART/SWV y **remota** mediante MQTT. Este planteamiento permite observar el estado del sistema en tiempo real tanto desde un terminal serie conectado a la placa como desde un cliente MQTT externo, facilitando la verificación de eventos clave como la conexión WiFi, el establecimiento de la sesión MQTT, las suscripciones, la recepción de comandos y los cambios de modo de funcionamiento del nodo.

3.7.1 Salida local: UART/SWV y control de concurrencia

Para la salida local se ha redirigido la salida estándar redefiniendo la función `_write()`, de manera que los mensajes generados por `printf()` se transmiten tanto por SWV (vía `ITM_SendChar`) como por USART1 (mediante `HAL_UART_Transmit`). Dado que el firmware está basado en tareas concurrentes, se ha incorporado un mutex de transmisión para evitar que distintas tareas impriman simultáneamente y se mezclen los mensajes. Además, se distingue entre ejecución en thread mode e ISR: en interrupciones se evita transmitir por UART y bloquear recursos (no se usan mutexes), preservando así el comportamiento temporal del sistema.

3.7.2 Uso de Mutex

En el sistema se ejecutan varias tareas en paralelo (gestión WiFi, conexión MQTT, adquisición del acelerómetro, configuración por UART, etc.), y todas ellas generan mensajes de depuración mediante `printf()`/`log_printf()`. Dado que la salida por UART es un recurso compartido, si varias tareas imprimen simultáneamente pueden producirse condiciones de carrera: caracteres intercalados, líneas mezcladas e incluso comportamientos no deterministas al acceder concurrentemente a `HAL_UART_Transmit()`.

Para evitarlo, se ha implementado un mecanismo de exclusión mutua basado en mutex (`uartTxMutexHandle` y `printMutexHandle`). En concreto, antes de realizar una salida por UART desde contexto de tarea, el código adquiere el mutex correspondiente (`osMutexAcquire`), garantizando que solo una tarea puede escribir en la UART en cada instante. Una vez completada la transmisión del mensaje, el mutex se libera (`osMutexRelease`). De este modo, cada mensaje se envía como un bloque coherente, manteniendo la trazabilidad y legibilidad de los logs durante la ejecución concurrente del sistema.

Además, se ha considerado el contexto de ejecución: cuando la impresión se realiza desde una interrupción (ISR), no se permite el uso de mutex ni operaciones potencialmente bloqueantes, ya que podría provocar un bloqueo del sistema. Por esta razón, en ISR se evita la transmisión por UART y se limita la salida a SWV (ITM), que es más ligera y no requiere sincronización bloqueante. Esta decisión mejora la robustez del sistema y asegura que las interrupciones mantengan un comportamiento temporal correcto.

En resumen, los mutex se han utilizado para proteger recursos compartidos (principalmente la UART) en un entorno multitarea, garantizando consistencia en los mensajes de depuración y evitando problemas derivados de accesos concurrentes.

3.7.3 Publicación remota: logs en MQTT sin realimentación

De forma complementaria se ha desarrollado una función (`log_printf`) que mantiene la salida local y, cuando existe conectividad, reenvía los logs a MQTT en un topic asociado al nodo: `bridge/log/<NODE_ID>` (por ejemplo `bridge/log/1` o `bridge/log/2`). Para evitar que la generación de logs dependa de la latencia de red, el envío no se realiza directamente, sino que el mensaje se encapsula y se introduce en la cola `qMqttTx`, quedando el `publish` delegado a la tarea MQTT. Adicionalmente, se han incorporado filtros para prevenir bucles de realimentación (logs de logs), evitando republicar mensajes relacionados con el propio topic de logs y habilitando el reenvío remoto únicamente cuando la conexión WiFi y la sesión MQTT se encuentran correctamente establecidas.

4 PRUEBAS Y VALIDACIÓN

4.1 Metodología de Desarrollo Incremental

4.1.1 Fase 1: Infraestructura de Comunicaciones (WiFi + MQTT)

La primera fase se centró en establecer la base de comunicaciones del sistema, implementando y probando únicamente las tareas `WifiTask` y `MQTT_Task` sin código de sensores. Los objetivos de validación en esta fase fueron:

Pruebas de conectividad WiFi:

- Verificación de inicialización correcta del módulo ISM43362
- Prueba de asociación a diferentes redes WiFi (entornos 2.4GHz congestionados y limpios)
- Validación de reconexión automática tras apagado/encendido del AP
- Medición de tiempos de establecimiento de conexión (típicamente 3-5 segundos)

Pruebas de cliente MQTT:

- Handshake exitoso con brokers públicos (`test.mosquitto.org`, `broker.emqx.io`)
- Publicación manual de mensajes de prueba (`test_topic`, payload "Hello MQTT")
- Verificación de recepción mediante cliente `mosquitto_sub` en PC
- Prueba de KeepAlive: desconectar cable de red del broker y verificar detección de timeout
- Suscripción a topic de control y confirmación de recepción de comandos desde `mosquitto_pub`

Scripts de validación Python:

En esta fase se desarrolló `test_monitor_env.py` simplificado (solo impresión de payloads recibidos, sin GUI) para confirmar que los mensajes MQTT llegaban correctamente al suscriptor.

Al finalizar esta fase, se disponía de una plataforma de comunicaciones robusta y validada que servía como fundación para las tareas de aplicación. Los logs del sistema confirmaban ciclos completos de conexión/desconexión/reconexión sin pérdidas de memoria ni cuelgues.

4.1.2 Fase 2: Integración de Sensores (Desarrollo Paralelo)

Una vez estabilizada la infraestructura de comunicaciones, se procedió al desarrollo de las tareas de sensores de forma paralela pero aislada, compilando versiones separadas del firmware según `NODE_ID`:

Desarrollo del Nodo Acelerómetro (`NODE_ID=1`)

Se implementó `Accel_Task` reutilizando las tareas WiFi/MQTT ya validadas. El desarrollo se centró en:

- Configuración del LSM6DSL: ODR 52Hz, FS $\pm 2g$, modo FIFO
- Prueba de interrupción FIFO watermark con nivel de 64 muestras
- Lectura correcta de muestras del eje Z y conversión a miligraves
- Construcción de mensajes JSON con metadata (t0, fs, s, n)
- Encolado en qMqttTx y verificación de publicación

Desarrollo del Nodo Ambiental (NODE_ID=2)

Se implementó task_envRead siguiendo el mismo patrón:

- Inicialización de sensores HTS221 (temperatura y humedad)
- Configuración de alarma RTC periódica (inicialmente 1 minuto para pruebas rápidas)
- Lectura de sensores y formateo JSON con campos id, temp, hum, origen
- Encolado y publicación vía MQTT_Task

Validación independiente:

Cada nodo se validó por separado mediante sus scripts Python específicos:

- test_accel.py: Reconstrucción de chunks JSON, verificación de secuencias (s, n)
- test_monitor_env.py: Decodificación de temperatura (división por 10) y humedad

Dos implementaciones funcionales independientes, cada una capaz de adquirir datos de su sensor y transmitirlos correctamente vía MQTT. Sin embargo, en esta fase surgió el problema crítico de Hard Fault descrito en la siguiente sección.

4.2 Debugging de Hard Fault por Stack Overflow

Durante las pruebas de la tarea del acelerómetro en modo continuo (1024 muestras), el sistema experimentaba resets inesperados tras enviar los primeros chunks de datos. El análisis del problema siguió el siguiente proceso de debugging sistemático.

4.2.1 Detección e Identificación

Durante la ejecución de la tarea del acelerómetro, el último log que se registraba era: [MQTT] Enviando Topic: bridge/accel/1... indicando que no se llegaba a transmitir ni un único chunk. Se plantearon los siguientes problemas:

- Factor 1: Buffer de red en MQTT_Task La librería coreMQTT declara un buffer estático NETWORK_BUFFER_SIZE (configurado en 1000 bytes en mqtt_priv.h) dentro de funciones de la pila de llamadas de MQTT_ProcessLoop(). Este buffer, sumado a los contextos MQTTContext_t y NetworkContext_t, consume aproximadamente 1400 bytes de stack.
- Factor 2: Buffer de payload en Accel_Task La tarea del acelerómetro declara un buffer local char payload[1024] para construir el JSON antes de encolarlo. Este buffer, más las variables de control de FIFO y los stack frames de las funciones llamadas, requiere aproximadamente 2200 bytes.

El overflow se manifestaba precisamente al publicar chunks porque:

1. Accel_Task construye el JSON en su buffer de 1024 bytes
2. Llama a send_mqtt_msg(), que encola en qMqttTx
3. MQTT_Task desencola y llama a prvMQTTPublishToTopic()
4. Esta función invoca la pila de serialización MQTT que usa el buffer de 1000 bytes
5. Si una interrupción ocurre en este punto (ej: SysTick), el procesador intenta hacer stacking y detecta que no hay espacio, generando el parón de la ejecución en el handler de hard fault.

4.2.2 Solución Implementada: Refactorización con Punteros

Tras identificar el stack overflow, se implementó una solución dual que combina optimización de memoria con aumento selectivo de stacks:

4.2.2.1 Estrategia 1: Refactorización de qMqttTx con punteros

Se reemplazó el patrón de copia por valor por gestión dinámica de memoria:

- La cola ahora almacena punteros (4 bytes) en lugar de estructuras completas (1060 bytes)
- `send_mqtt_msg()` aloca memoria con `pvPortMalloc()` solo cuando se va a enviar un mensaje
- `MQTT_Task` consume el mensaje y libera con `vPortFree()` tras publicar
- Resultado: Reducción de footprint de cola de 8480 → 32 bytes

4.2.2.2 Estrategia 2: Ajuste de tamaños de stack

- `MQTT_Task`: 5120 → 4096 bytes (reducción gracias a eliminación de variable local `MqttMsg_t`)
- `Accel_Task`: Mantenido en 4096 bytes (requiere buffer de construcción de JSON)
- Heap total: Aumentado a 24000 bytes para soportar alocações dinámicas

Validación:

Se realizaron pruebas de estrés con modo continuo (16 chunks × 1024 bytes), verificando:

- Ausencia de Hard Faults durante la realización de toda la prueba
- Uso máximo de heap: aproximadamente 18500 bytes (77% del total, margen adecuado)
- Logs de debug con direcciones heap (`[MALLOC] 0x20001450`, `[FREE] 0x20001450`) confirmando correcto ciclo malloc/free sin leaks


```

[BOOT] Forzando Reinicio Fisico del WiFi ---
[BOOT] WiFi Reiniciado. Iniciando Kernel... ---
[UART] RX listo

=====
      MENU DEBUG - COMANDOS DISPONIBLES
=====
UART (terminal) y MQTT (bridge/cmd/<id>)

Acelerometro:
CONT ON           -> modo continuo
CONT OFF          -> modo normal
READ              -> forzar lectura

WiFi:
WIFI <ssid> <pass> -> cambiar credenciales
ej: WIFI MiRed 12345678

RTC:
RTC 2026-02-08 19:30:00
RTC 2026 2 8 19 30 0

MQTT topic de control:
bridge/cmd/1 (nodo 1)
bridge/cmd/2 (nodo 2)
=====

[ACC] Task start (FIFO)
--- [WIFI] Tarea iniciada ---
[WIFI] llamando a wifi_connect()...
ES-WIFI Initialized.
> eS-WiFi module MAC Address : C4:7F:51:92:98:B2

Connecting to manolo, 123456789

```

```
> es-wifi module connected: got IP Address : 10.72.214.112  
[WiFi] Conexion Exitosa.  
--- [MQTT] Tarea Iniciada ---  
[MQTT] Conectando al Broker...  
Create a TCP connection to broker.emqx.io:1883.  
MQTT initialized  
MQTT connected to broker  
[MQTT] Loop de transmision activo.  
Subscription to bridge/cmd/1, result: 0, success  
[MQTT] Subscrito a: bridge/cmd/1  
[MALLOC] msg en heap = 0x200059f0  
[MQTT] Enviando Topic: bridge/log/1...  
Topic "bridge/cmd/1": publicado "READ"  
[ACC] Capture start FIFO mode=NORMAL target=64  
[MALLOC] msg en heap = 0x200059f0  
[MQTT] Enviando Topic: bridge/log/1...  
[ACC][JSON] {"t0":35503,"fs":52,"s":0,"n":1,"z":  
[1055,1057,1055,1055,1056,1054,1055,1056,1055,1055,1056,1055,1054,  
[MALLOC] msg en heap = 0x200059f0  
[MALLOC] msg en heap = 0x20005e18  
[ACC] Done FIFO. collected=64 chunks=1  
[MALLOC] msg en heap = 0x20006240  
[MQTT] Enviando Topic: bridge/log/1...  
[MQTT] Enviando Topic: bridge/accel/1...  
[MALLOC] msg en heap = 0x200059f0  
Published to topic bridge/accel/1: {"t0":35503,"fs":52,"s":0,"n":1,"z":  
[1055,1057,1055,1055,1056,1054,1055,1056,1055,1055,1056,1055,1054,  
5,1056,1055,1055,1056,1054,1056,1056,1055,1055,1056,1054,1056,1055,1054,105  
[MQTT] Enviando Topic: bridge/log/1...  
[MQTT] Enviando Topic: bridge/log/1...
```

Figura 5: algunos logs de la ejecución del nodo Acelerómetro

5 INTERFAZ DE USUARIO

Para la representación de los datos se ha creado una interfaz gráfica, usando la librería Tkinter. La interfaz incluye cuatro cuadros, para la temperatura, humedad, aceleración, y transformada de fourier de la aceleración. Esto último se calcula para detectar oscilaciones en el puente, lo que puede ser crítico para predecir el riesgo de derrumbe. Además de todo esto, la interfaz incluye una serie de botones, para que el usuario simplemente tenga que pulsar un botón. Los comandos implementados son:

- Forzar medida
- Activar alarma
- Desactivar alarma
- Sincronizar relojes

Los comandos se mandan a todas las placas, ya que se entiende que todas miden el mismo puente, y no hay necesidad de discriminar parejas. Al pulsar el comando de sincronizar relojes, estos se configuran con la hora local del equipo que ha emitido la orden.

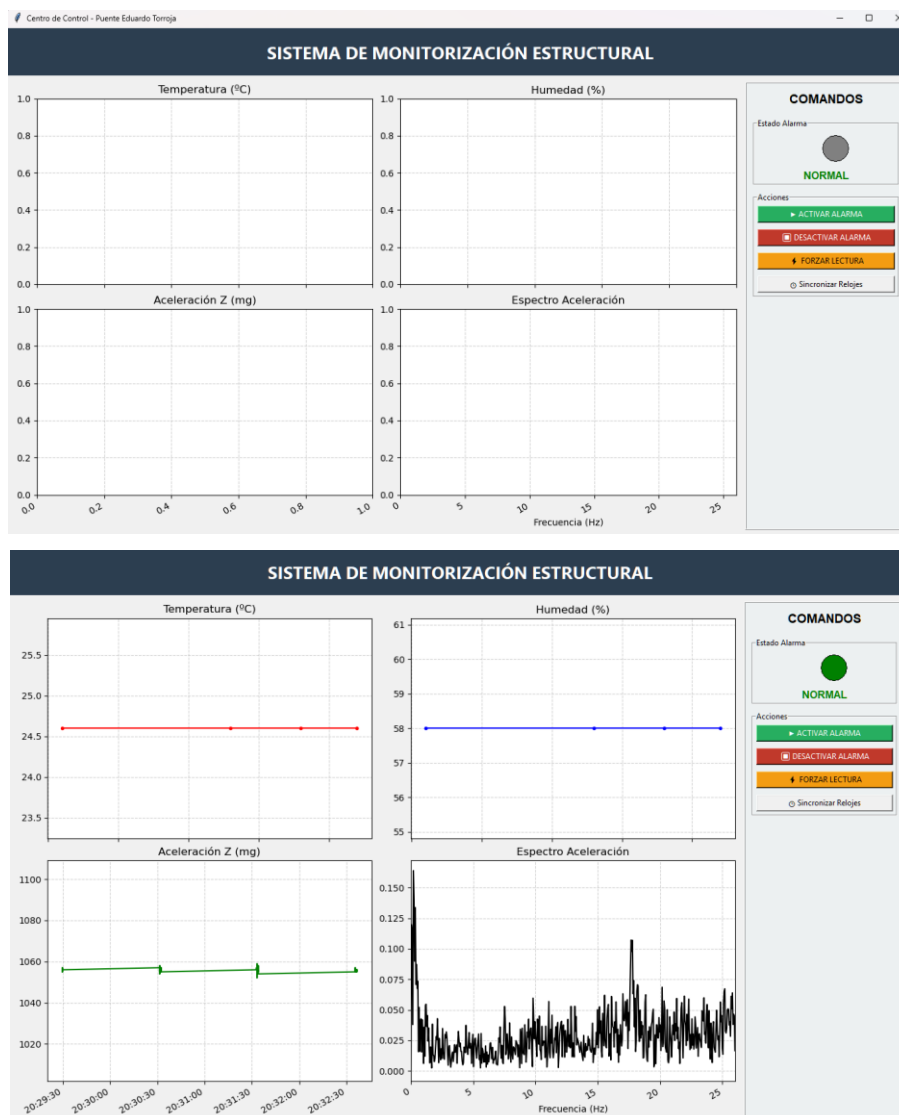


Figura 6. Interfaz gráfica.

6 CONCLUSIONES Y TRABAJOS FUTUROS

6.1 Conclusiones

El trabajo realizado permite validar que una arquitectura basada en FreeRTOS + WiFi + MQTT es adecuada para construir un sistema IoT modular en un entorno embebido.

Desde el punto de vista del diseño, la decisión de estructurar el firmware por roles parametrizados mediante `NODE_ID` y topics MQTT por identificador demuestra ser un enfoque escalable: la lógica principal no depende de un caso particular de nodo, sino de una convención de direccionamiento (`bridge/.../<id>`). Esto permite pensar en el sistema como una “red” en la que cada nodo se configura y se controla de forma homogénea. Además, el control distribuido pone de manifiesto que MQTT no se utiliza solo como canal de telemetría, sino también como mecanismo de coordinación del sistema.

En el caso del nodo acelerómetro, el uso de la FIFO aporta una ventaja clara frente al sondeo directo: reduce sensibilidad a jitter del sistema (por ejemplo, cuando la CPU está ocupada por comunicaciones) y favorece que los datos se capturen en bloques consistentes, lo cual es especialmente importante cuando se requiere muestreo continuo o se desea minimizar carga del microcontrolador. En otras palabras, se consigue desacoplar el ritmo de adquisición del ritmo de envío, lo que mejora la robustez general del nodo.

Como principal limitación, el sistema todavía está orientado a un entorno de laboratorio: la conectividad se basa en MQTT sin mecanismos avanzados de seguridad ni persistencia de configuración. Asimismo, el formato JSON facilita la depuración pero introduce sobrecarga, y la gestión de errores de red podría endurecerse para escenarios reales con cortes frecuentes. Aun así, el proyecto cumple el objetivo de demostrar un sistema IoT completo y controlable en ejecución, y deja una base sólida para evolucionar hacia una solución más robusta y desplegable.

6.2 Trabajos futuros

Las principales mejoras identificadas se orientan a robustez, escalabilidad y calidad de sistema:

- Mayor robustez en red: incorporar políticas de reconexión más avanzadas (backoff exponencial, re-suscripción controlada, detección de sesión inválida) y mecanismos de diagnóstico del enlace.
- Seguridad en comunicaciones: migrar de MQTT plano a MQTTS (TLS) y añadir autenticación, especialmente si el sistema se usa fuera de un entorno de laboratorio
- Optimización de mensajes: ajustar el tamaño de bloque del acelerómetro según la carga del sistema, reducir overhead del JSON (por ejemplo usando CBOR o formatos compactos) y añadir validación de integridad (CRC o checksum) si se quiere mayor fiabilidad
- Escalabilidad real a N nodos: formalizar un esquema de registro de nodos y ampliar la lógica de topics/handlers para poder incorporar nuevos roles sin modificar la estructura principal del firmware
- Inclusión de selector de nodos en la interfaz: si bien los comandos se envían a todos los nodos, es conveniente poder seleccionar una pareja para ver sus gráficas correspondientes. Para esto se podría asignar un identificador no al nodo, sino a la pareja de nodos